

TN SKILLS

Project Documentation Report

A Project Report on

ComicCraft

AI Comic Story Creator Using Gemini Models

Submitted in partial fulfilment of the requirements of the
TN Skills Portal Submission

Submitted by

Thamizharasi S (222502807)

Nasrin N (222502792)

Mahadharshini B (222502789)

Shanmugapriya K (222502803)

Vanthana K (222502808)

Academic Year 2026

Repository: github.com/balsslaba7-cpu/ComicCraft — AI Comic Story Creator using Gemini Models

Bonafide Certificate

This is to certify that the project report entitled "ComicCraft – AI Comic Story Creator Using Gemini Models" is a bonafide record of work carried out by Thamizharasi S (222502807), Nasrin N (222502792), Mahadharshini B (222502789), Shanmugapriya K (222502803), and Vanthana K (222502808), submitted in partial fulfilment of the requirements of the TN Skills Portal Submission under the TN Skills initiative, during the Academic Year 2026.

The work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion, and does not form part of any other project report on the basis of which a degree or award is conferred on a future occasion.

Faculty Mentor / Project Guide

TN Skills Initiative – TN Skills Portal

Place: Tamil Nadu

Date: 2026

Acknowledgement

We express our sincere gratitude to the TN Skills initiative for the opportunity to design, build and document this project. Working on ComicCraft allowed us to explore the practical application of Generative AI models to a creative domain, while strengthening our understanding of rapid application development using Python and Streamlit.

We would like to thank our faculty mentors and project guides for their valuable feedback and encouragement throughout the development of this project.

We also acknowledge the developers of Streamlit, the Google GenAI SDK, and the Gemini model family, whose tools and platforms made the rapid development of this application possible.

Finally, we thank our peers and family for their continued support and encouragement.

Abstract

Visual storytelling has traditionally required a combination of creative writing skill and artistic ability, making comic creation inaccessible to many aspiring storytellers. ComicCraft is proposed as an AI-powered comic story creator that removes this barrier by using Google's Gemini Large Language Model to automatically generate structured, five-panel comic stories from a simple user idea or theme, complete with scene descriptions, character actions, character dialogues, and image prompts suitable for downstream illustration.

The application is built as a lightweight, single-service Python application using Streamlit for the user interface and the Google GenAI SDK for communicating with the Gemini API. Given a story idea or prompt entered by the user, ComicCraft orchestrates a structured generation pipeline that produces a coherent five-panel narrative: each panel includes a scene setting, the actions performed by the characters within that scene, the dialogue or captions spoken by those characters, and a descriptive image prompt that can be used to generate corresponding artwork. A character consistency mechanism ensures that character names, appearances, and personalities remain coherent across all five panels of the generated story.

This report documents the design, architecture, technology stack, implementation, and testing of the ComicCraft application. It also discusses the feasibility of the system, its advantages and limitations, and potential directions for future enhancement, such as direct image generation integration and multi-format export.

The project demonstrates how a modern Generative AI model can be combined with a minimal, easy-to-deploy web framework such as Streamlit to build a creative, user-friendly tool that lowers the barrier to entry for visual storytelling and comic creation.

Table of Contents

Bonafide Certificate	2
Acknowledgement	3
Abstract	4
Chapter 1: Introduction	8
1.1 Overview	8
1.2 Problem Statement	8
1.3 Objectives	8
1.4 Scope of the Project	8
1.5 Organization of the Report	9
Chapter 2: Literature Survey	10
2.1 General-Purpose AI Writing Assistants	10
2.2 Comic and Storyboard Generation Research	10
2.3 Low-Code / No-Code Creative Tools	10
2.4 Identified Gaps	10
2.5 Comparative Summary	10
Chapter 3: System Analysis	12
3.1 Proposed System	12
3.2 Feasibility Study	12
3.3 Software and Hardware Requirements	12
3.4 Requirement Specification	13
3.5 Development Methodology	13
Chapter 4: System Architecture and Design	15
4.1 Architecture Overview	15
4.2 Component-wise Architecture Description	15
4.3 Data Flow	16
4.4 Use Case Description	16
4.5 Sequence of Operations	17
Chapter 5: Technology Stack	18
Chapter 6: Implementation and Setup	19
6.1 Environment Configuration	19
6.2 Installation and Launch Steps	19
6.3 Application Internals	20
Chapter 7: Feature Description	21
7.9 Sample Walkthrough	22
Chapter 8: Testing	23
8.1 Levels of Testing	23
8.2 Test Cases	23

Chapter 9: Results, Discussion, Conclusion and Future Enhancements	25
9.1 Results and Discussion	25
9.2 Advantages	25
9.3 Limitations	25
9.4 Conclusion	26
9.5 Future Enhancements	26
References	27
Appendix A: Sample requirements.txt	28
Appendix B: Glossary	28
Appendix C: Illustrative Prompt Template	29
Appendix D: Frequently Asked Questions	29

List of Figures and Tables

Figure 4.1: High-Level System Architecture of ComicCraft

Table 3.1: Software Requirements

Table 3.2: Hardware Requirements

Table 8.1: Test Cases and Results

Chapter 1: Introduction

1.1 Overview

Comics and graphic storytelling are a powerful medium for communicating ideas, emotions, and narratives through a combination of visuals and text. However, creating a comic story traditionally requires both narrative writing skills and the ability to plan out sequential visual panels — a combination of skills that many people, especially students, hobbyists and early-stage creators, do not possess.

ComicCraft addresses this challenge by using Google's Gemini Large Language Model to automatically transform a simple story idea into a structured, five-panel comic narrative. The application handles the creative heavy-lifting — scene composition, character behavior, dialogue writing, and visual prompt generation — allowing any user to produce a complete comic story concept within seconds, through a simple and intuitive Streamlit-based web interface.

1.2 Problem Statement

Aspiring storytellers, students, and content creators frequently have an idea for a story but lack either the time, writing expertise, or artistic ability to translate that idea into a structured comic format. Manually planning a multi-panel comic — deciding what happens in each scene, how characters behave and speak, and what each panel should visually depict — is a time-consuming and skill-intensive process.

Additionally, most existing AI writing tools generate a single block of prose rather than a structured, panel-by-panel narrative suitable for comic adaptation, and they typically do not maintain character consistency across multiple generated segments, resulting in narratives where a character's name, appearance or personality shifts inconsistently between scenes.

1.3 Objectives

- To design and implement a Streamlit-based web application that allows users to generate a complete five-panel comic story from a simple text prompt.
- To integrate Google's Gemini model, via the Google GenAI SDK, to perform the underlying natural language generation.
- To structure the AI-generated output into distinct, well-defined components for each panel: scene description, character actions, character dialogue, and image prompt.
- To implement a character consistency mechanism that preserves character identity, appearance and personality across all panels of a generated story.
- To provide a simple mechanism for users to download their generated comic story for offline use or further development.

- To securely manage the Gemini API credentials using environment variables via `python-dotenv`.

1.4 Scope of the Project

The scope of this project covers the design and development of the ComicCraft application as a single-service Python/Streamlit tool that generates textual comic story content — scenes, actions, dialogues, and image prompts — using the Gemini model. The current implementation focuses on text-based story generation and does not include automatic image generation or rendering of the actual comic artwork; the generated image prompts are intended for use with a separate image-generation tool or artist.

Multi-user account management, story persistence in a database, and collaborative editing features are outside the scope of the current version and are discussed as potential future enhancements in the concluding chapter of this report.

1.5 Organization of the Report

This report is organized into nine chapters. Chapter 1 introduces the project. Chapter 2 surveys related work in AI-assisted creative writing and comic generation. Chapter 3 discusses system analysis, including feasibility. Chapter 4 presents the system architecture and design, including a system diagram. Chapter 5 describes the technology stack. Chapter 6 details the implementation and setup procedure. Chapter 7 describes the core features of the application. Chapter 8 covers testing methodology. Chapter 9 presents results, discussion, conclusion and future enhancements, followed by references.

Chapter 2: Literature Survey

Prior to designing ComicCraft, a review of existing AI-assisted creative writing and story generation tools was conducted to identify design opportunities and gaps that this project could address.

2.1 General-Purpose AI Writing Assistants

General-purpose AI writing tools built on large language models can generate creative prose, including short stories, when given a prompt. However, these tools typically produce a single continuous block of narrative text rather than a structured, panel-by-panel breakdown suitable for direct adaptation into a comic format, requiring the user to manually segment and re-interpret the output.

2.2 Comic and Storyboard Generation Research

Academic and industry research into automated storyboard and comic generation has explored combining language models with image generation models to produce fully illustrated comics. While promising, many such systems are research prototypes requiring significant computational resources and are not readily accessible as lightweight, easy-to-deploy applications for individual creators.

2.3 Low-Code / No-Code Creative Tools

Web-based creative tools built with frameworks such as Streamlit and Gradio have become popular for rapidly prototyping AI-powered applications, as they allow developers to build functional user interfaces around a machine learning or LLM backend with minimal frontend code. This trend directly informed the choice of Streamlit as the interface framework for ComicCraft, prioritizing rapid development and ease of deployment over a fully custom frontend.

2.4 Identified Gaps

The survey identified the following gaps:

1. A lack of lightweight, easily deployable tools that generate structured, panel-based comic narratives rather than plain prose.
2. Limited attention to character consistency across multiple generated narrative segments in existing lightweight tools.
3. Minimal focus on generating structured image prompts alongside narrative text, which would bridge the gap between story generation and eventual visual illustration.

ComicCraft addresses these gaps by using a structured generation approach that explicitly separates each panel into scene, action, dialogue and image-prompt components, while

incorporating a dedicated character consistency mechanism, all within a minimal, easy-to-run Streamlit application.

2.5 Comparative Summary

Table 2.1 summarizes how ComicCraft's design choices compare against the categories of existing tools surveyed above.

Category	Typical Output	Character Consistency	Deployment Effort
General-purpose AI writing assistants	Single block of prose	Not guaranteed	Low
Research storyboard/comic generators	Illustrated multi-panel comics	Varies	High (GPU/infra needed)
Low-code prototyping tools (generic)	Depends on developer's build	Depends on developer's build	Low to Medium
ComicCraft (this project)	Structured 5-panel text + image prompts	Explicitly engineered	Low

Table 2.1: Comparative Summary of Related Approaches

Chapter 3: System Analysis

3.1 Proposed System

The proposed system, ComicCraft, is a single-service Python application that uses Streamlit to present a simple web-based interface where a user enters a comic story idea or theme. This input is sent to the Google Gemini model via the Google GenAI SDK, using a carefully constructed prompt that instructs the model to produce a five-panel comic story broken down into scene descriptions, character actions, character dialogues, and image prompts for each panel, while maintaining consistent characters throughout.

The generated story is displayed within the Streamlit interface in a readable, panel-by-panel format, and the user is given the option to download the complete generated story as a text file for offline use, sharing, or as a reference for an illustrator or image-generation tool.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The technology stack used in ComicCraft — Python, Streamlit, and the Google GenAI SDK — is well-documented, actively maintained, and widely used in the rapid AI-application development community. Streamlit's simple, declarative API allows a fully functional web interface to be built with a fraction of the code required by a traditional frontend framework, making the project technically feasible for a small development effort.

3.2.2 Economic Feasibility

ComicCraft relies on open-source software (Python, Streamlit, python-dotenv) and the Google Gemini API, which offers a free usage tier suitable for development, demonstration, and light personal use. There are no licensing costs associated with the technologies used, and hosting a Streamlit application is inexpensive, with several free hosting options available for small-scale deployment (such as Streamlit Community Cloud), making the project highly economical.

3.2.3 Operational Feasibility

The application's single-page, form-based interface requires no training for end users — a user simply enters a story idea and receives a generated comic story. The minimal operational footprint (a single Python script and a small set of dependencies) also makes the application easy to maintain, update, and redeploy.

3.3 Software and Hardware Requirements

Requirement	Specification
Operating System	Windows 10/11, macOS, or Linux

Runtime	Python 3.9 or later
Web Framework	Streamlit
AI SDK	Google GenAI SDK (google-genai / google-generativeai)
Utility Library	python-dotenv
Development Tools	Visual Studio Code or any Python IDE
Version Control	Git and GitHub

Table 3.1: Software Requirements

Requirement	Specification
Processor	Intel Core i3 / AMD equivalent or higher (minimum)
RAM	4 GB minimum, 8 GB recommended
Storage	Less than 500 MB for dependencies and application files
Internet Connectivity	Required for Gemini API calls

Table 3.2: Hardware Requirements

3.4 Requirement Specification

3.4.1 Functional Requirements

- The system shall accept a free-text comic story idea or theme from the user through a Streamlit input field.
- The system shall construct a structured prompt and send it to the Gemini model via the Google GenAI SDK.
- The system shall parse the model's response into exactly five panels, each with scene, action, dialogue and image-prompt fields.
- The system shall display the parsed five-panel story within the Streamlit interface in a readable format.
- The system shall allow the user to download the generated story as a text file.
- The system shall display an informative error message if the GEMINI_API_KEY is missing, invalid, or the network is unavailable.

3.4.2 Non-Functional Requirements

- Usability: the interface shall require no technical training and shall be operable by a first-time user within seconds.
- Performance: a five-panel story shall typically be generated and displayed within the response time of a single Gemini API call.
- Security: the GEMINI_API_KEY shall never be hard-coded in source files or committed to version control.
- Portability: the application shall run unmodified on Windows, macOS, and Linux, given a compatible Python runtime.
- Maintainability: the single-file design shall keep the codebase easy to read,

debug, and extend.

3.5 Development Methodology

The project followed a Rapid Application Development (RAD) approach, well suited to a small, single-service application built around a rapidly evolving AI model. Given the lightweight nature of the Streamlit framework, the development cycle emphasized quick prototyping of the prompt design and UI layout, followed by iterative refinement based on the quality and structure of the AI-generated comic stories.

Each iteration involved refining the prompt sent to the Gemini model to improve the consistency, structure, and creative quality of the generated five-panel stories, testing the change interactively within the Streamlit interface, and incorporating improvements based on observed output quality.

Chapter 4: System Architecture and Design

4.1 Architecture Overview

ComicCraft follows a simple, single-service architecture centered around a Streamlit application that directly interfaces with the Google Gemini model through the Google GenAI SDK. Despite its architectural simplicity, the internal story-generation logic is organized into clearly defined conceptual stages to ensure structured, consistent output. Figure 4.1 illustrates the high-level architecture of the system.

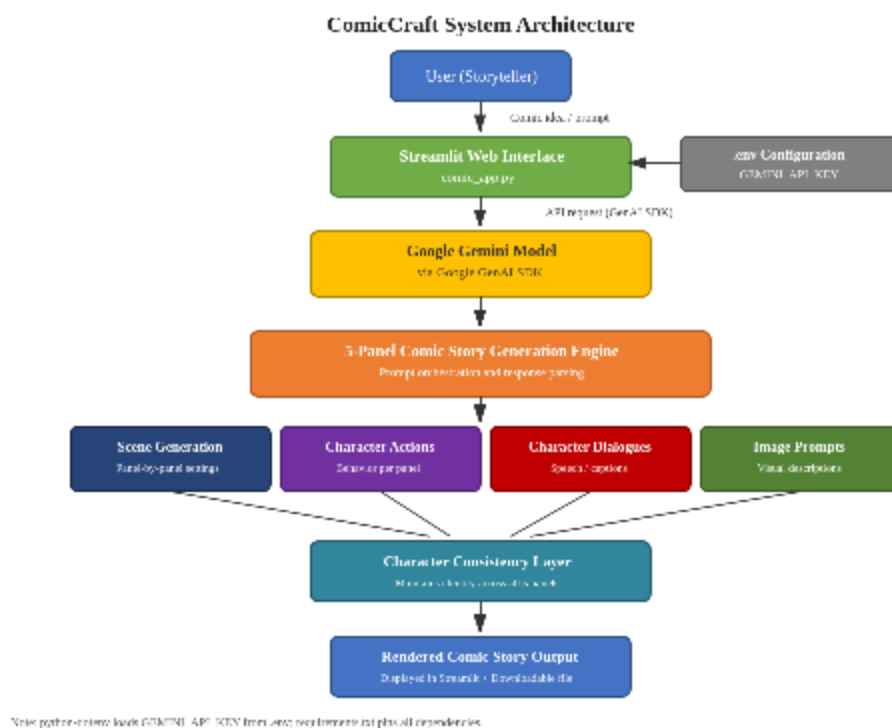


Figure 4.1: High-Level System Architecture of ComicCraft

4.2 Component-wise Architecture Description

4.2.1 Presentation Layer (Streamlit Interface)

The presentation layer is implemented entirely within `comic_app.py` using Streamlit. It renders the input form where the user enters their comic story idea, displays the generated five-panel story in a readable format, and provides a download button for exporting the result. Streamlit's reactive execution model means the entire script re-runs on user interaction, simplifying state management for this relatively lightweight application.

4.2.2 Configuration Layer (.env)

The `.env` file, loaded via `python-dotenv`, securely stores the `GEMINI_API_KEY` required to authenticate with the Google Gemini API. Keeping this credential outside of the source code

prevents accidental exposure of the API key in version control and allows different environments (development, demonstration) to use different keys without code changes.

4.2.3 AI Integration Layer (Google GenAI SDK)

The AI integration layer uses the Google GenAI SDK to construct and send a structured prompt to the Gemini model based on the user's story idea, and to receive the generated response. This layer is responsible for formatting the prompt to explicitly request a five-panel structure with scene, action, dialogue and image-prompt components for each panel.

4.2.4 Story Generation Engine

The Story Generation Engine is the conceptual core of ComicCraft's logic within `comic_app.py`. It takes the raw response from the Gemini model and parses it into five distinct panels, each containing a scene description, the actions performed by characters, the dialogue spoken, and an image prompt describing the visual content of that panel. This structured parsing ensures the output can be consistently displayed and formatted regardless of minor variations in the raw model response.

4.2.5 Character Consistency Layer

To prevent characters from being described inconsistently across the five panels (for example, a character's name or appearance changing partway through the story), the prompt design instructs the Gemini model to establish character identities at the outset and refer back to them consistently. This is reinforced through prompt engineering techniques such as explicitly listing established characters and their traits before requesting each subsequent panel.

4.3 Data Flow

1. The user enters a comic story idea or theme into the Streamlit input form.
2. On submission, `comic_app.py` constructs a structured prompt instructing the Gemini model to generate a five-panel comic story with scene, action, dialogue and image-prompt components while maintaining character consistency.
3. This prompt is sent to the Gemini model via the Google GenAI SDK, authenticated using the `GEMINI_API_KEY` loaded from the `.env` file.
4. The model's response is received and parsed into five structured panels by the Story Generation Engine.
5. The parsed story is rendered within the Streamlit interface, and the user is given the option to download the complete story as a text file.

Because ComicCraft does not persist data to an external database, each session is stateless beyond the current Streamlit session state — every new story idea submitted results in an independent generation cycle, which simplifies the overall architecture considerably compared to a multi-user, persistent web application.

4.4 Use Case Description

The primary actor in the ComicCraft system is the User (a storyteller, student, or hobbyist creator). The key use cases supported include: Enter Story Idea, Generate Five-Panel Comic Story, View Generated Story, and Download Story. Each use case is realized through direct interaction with the single-page Streamlit interface, which internally coordinates the call to the Gemini API and the subsequent parsing and display of the result.

For example, in the 'Generate Five-Panel Comic Story' use case, the user submits a story idea; the application constructs and sends the corresponding prompt to Gemini; the response is parsed into scene, action, dialogue and image-prompt fields for each of the five panels; and the complete structured story is displayed to the user within the same interface, without requiring a page reload.

4.5 Sequence of Operations

Table 4.1 presents the sequence of operations that occurs during a single story-generation cycle, from user input to displayed output.

Step	Actor / Component	Action
1	User	Enters a comic story idea into the Streamlit text input.
2	Streamlit Interface	Captures the input on form submission and triggers generation.
3	Configuration Layer	Supplies the GEMINI_API_KEY loaded from .env.
4	AI Integration Layer	Builds the structured prompt and calls the Gemini model via the GenAI SDK.
5	Google Gemini Model	Generates free-form text covering five panels.
6	Story Generation Engine	Parses the response into scene, action, dialogue and image-prompt fields.
7	Character Consistency Layer	Reinforces consistent character identity across the parsed panels.
8	Streamlit Interface	Renders the five panels and offers a download button.

Table 4.1: Sequence of Operations for a Story-Generation Cycle

Chapter 5: Technology Stack

ComicCraft is intentionally built using a minimal but effective technology stack, prioritizing rapid development, ease of deployment, and straightforward maintenance. This chapter describes each of the core technologies used.

5.1 Frontend / Interface

- **Streamlit:** A Python framework for rapidly building interactive, data- and AI-driven web applications with minimal code, used here to build the entire user interface of ComicCraft, including the input form, story display, and download functionality — all within a single Python script.

5.2 Core Application

- **Python:** The core programming language used to implement the entire application logic, including prompt construction, API communication, and response parsing.
- **python-dotenv:** Used to load environment variables, specifically the `GEMINI_API_KEY`, from a local `.env` file, keeping sensitive credentials out of the source code.

5.3 Artificial Intelligence Stack

- **Google Gemini API:** The underlying Large Language Model used to generate the five-panel comic story content, including scene descriptions, character actions, dialogues, and image prompts, based on a structured prompt derived from the user's input idea.
- **Google GenAI SDK:** The official Python SDK used to authenticate with and send requests to the Google Gemini model, and to receive and process the generated text response within the application.

5.4 Data Persistence

- **ComicCraft** does not use a persistent database in its current version. Generated stories exist only within the active Streamlit session and are made available to the user via a direct file download, rather than being stored server-side.

5.5 Development Tools

- **requirements.txt:** Used to declare and pin all Python package dependencies, ensuring that the application can be reliably installed in a new environment with a single `pip` command.
- **Git and GitHub:** Used for version control and source code management throughout the development of the project.
- **Visual Studio Code:** Used as the primary development environment for writing and debugging `comic_app.py`.

Chapter 6: Implementation and Setup

This chapter describes the practical setup and implementation procedure for running ComicCraft, based on the project's structure of a single Streamlit application (`comic_app.py`), a `requirements.txt` dependency manifest, and a `.env` configuration file.

6.1 Environment Configuration

Before running the application, a `.env` file must be created in the root of the ComicCraft project directory. This file must define the `GEMINI_API_KEY` variable, containing a valid API key obtained from Google's Generative AI platform. This key is loaded at runtime using `python-dotenv` and used to authenticate all requests made to the Gemini model, ensuring that the credential is never hard-coded into `comic_app.py` or committed to version control.

```
# .env
GEMINI_API_KEY=<your_api_key_here>
```

6.2 Installation and Launch Steps

1. Clone or download the repository: obtain the ComicCraft project files, including `comic_app.py`, `requirements.txt`, and `README.md`, either by cloning the GitHub repository or downloading the project archive.
2. Create a virtual environment (recommended): a dedicated Python virtual environment is created for the project using `"python -m venv env"` to isolate ComicCraft's dependencies from the system-wide Python installation.
3. Activate the virtual environment: the environment is activated (for example, using `"./env/Scripts/activate"` on Windows or `"source env/bin/activate"` on macOS/Linux) so that subsequent package installations are scoped to the project.
4. Install dependencies: running `"pip install -r requirements.txt"` installs all required Python packages, including Streamlit, the Google GenAI SDK, and `python-dotenv`, as declared in the project's dependency manifest.
5. Configure the `.env` file: the developer creates a `.env` file in the project root and adds the line `"GEMINI_API_KEY=<your_api_key_here>"`, replacing the placeholder with a valid Gemini API key.
6. Launch the application: running `"streamlit run comic_app.py"` starts the Streamlit development server and automatically opens the ComicCraft interface in the default web browser, typically accessible at `http://localhost:8501`.
7. Generate a comic story: once the application is running, the user enters a comic story idea or theme into the provided input field and submits the form, triggering the application to call the Gemini API and display the generated five-panel comic story.
8. Download the result: after a story is generated, the user can click the download option provided in the interface to save the complete comic story as

a text file for offline use.

```
git clone https://github.com/balssslaba7-cpu/ComicCraft.git
cd ComicCraft
python -m venv env
source env/bin/activate    # Windows: env\Scripts\activate
pip install -r requirements.txt
echo GEMINI_API_KEY=<your_api_key_here> > .env
streamlit run comic_app.py
```

6.3 Application Internals

Internally, `comic_app.py` is organized around a small number of core responsibilities: rendering the Streamlit UI components (title, input field, submit button, and output display area); constructing a detailed prompt string that instructs the Gemini model to produce a five-panel story with explicit scene, action, dialogue and image-prompt sections while preserving character consistency; invoking the Google GenAI SDK to send this prompt and receive the model's response; parsing the returned text into five structured panels; and rendering the parsed panels back into the Streamlit interface using appropriately formatted headers and text blocks for readability.

This straightforward, single-file design keeps the application easy to understand, debug, and extend, which is well suited to the project's goal of providing a lightweight, accessible comic-story generation tool rather than a large-scale, multi-service system.

6.3.1 Illustrative Workflow (Pseudocode)

The following simplified pseudocode illustrates the conceptual flow described above; it is representative of the described logic rather than a verbatim reproduction of the source file.

```
import streamlit as st
from dotenv import load_dotenv
from google import genai
import os
load_dotenv()
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
st.title("ComicCraft - AI Comic Story Creator")
idea = st.text_input("Enter your comic story idea:")
if st.button("Generate Comic") and idea:
    prompt = build_five_panel_prompt(idea) # scene/action/dialogue/image-prompt
    response = client.models.generate_content(
        model="gemini-2.0-flash", contents=prompt
    )
    panels = parse_into_five_panels(response.text)
    render_panels(panels)                # display in Streamlit
    st.download_button("Download Story", format_story(panels))
```

Chapter 7: Feature Description

This chapter describes the core features of ComicCraft in detail, building on the architectural description provided in Chapter 4.

7.1 AI-Powered Five-Panel Comic Story Generation

The central feature of ComicCraft is its ability to transform a short user-provided idea or theme into a complete, structured five-panel comic story. Rather than producing a single block of prose, the system explicitly requests and parses five distinct panels, giving the output a clear narrative arc suitable for direct adaptation into visual comic form.

7.2 Scene Generation

For each of the five panels, ComicCraft generates a distinct scene description that sets the location, time, and visual context for that part of the story, ensuring that the narrative progresses logically from one panel to the next.

7.3 Character Actions

Alongside each scene description, the system generates a description of the actions performed by the characters present in that panel, helping to convey the story's physical events and character motivations visually and narratively.

7.4 Character Dialogues

ComicCraft generates dialogue or captions for the characters in each panel, giving voice to the story and allowing readers to follow the characters' thoughts, conversations, and reactions throughout the five-panel narrative.

7.5 Image Prompt Generation

For each panel, the system also generates a descriptive image prompt summarizing the visual content of that scene — including setting, characters, and action — which can be directly used with an external image-generation tool or as a brief for a human illustrator, bridging the gap between the generated narrative and eventual visual artwork.

7.6 Character Consistency

A dedicated character consistency mechanism, implemented through careful prompt design, ensures that character names, appearances, and personality traits remain stable across all five panels, avoiding the inconsistencies commonly seen in naively-generated multi-part AI narratives.

7.7 Downloadable Output

Once a comic story has been generated, users can download the complete five-panel narrative — including all scene descriptions, actions, dialogues, and image prompts — as a text file, allowing them to save, share, or further develop their comic story offline.

7.8 Simple, User-Friendly Interface

The entire application is accessible through a single, uncluttered Streamlit page, requiring no technical knowledge from the end user beyond typing a story idea into a text box and clicking a button, making the tool accessible to students, hobbyists, and creative writers alike.

7.9 Sample Walkthrough

The following illustrative walkthrough demonstrates the expected output shape for a sample story idea, consistent with the structured format described in Sections 7.1 to 7.5.

User Input: "A curious fox who discovers a hidden library in the forest."

Panel 1

Scene: A misty forest clearing at dawn.

Action: Vix the fox sniffs at a moss-covered stone archway.

Dialogue: Vix: "That wasn't here yesterday..."

Image Prompt: A red fox beside an ancient stone archway in a misty forest, dawn light.

Panel 2

Scene: Inside a dim, dusty underground library.

Action: Vix pushes open a heavy wooden door and peers inside.

Dialogue: Vix: "Books... hundreds of them."

Image Prompt: A small fox silhouetted in a doorway facing towering bookshelves.

Panel 3 - Panel 5 follow the same structure, continuing the narrative while keeping Vix's name, appearance and personality consistent.

As shown above, each panel consistently refers to the same character ("Vix"), demonstrating the effect of the character consistency mechanism described in Section 4.2.5.

Chapter 8: Testing

Testing for ComicCraft focused on validating both the functional correctness of the Streamlit application and the qualitative structure and consistency of the AI-generated comic stories, given the inherently non-deterministic nature of Large Language Model output.

8.1 Levels of Testing

8.1.1 Functional Testing

Functional testing verified that the Streamlit interface correctly accepts user input, that the application correctly constructs and sends prompts to the Gemini API, that valid responses are correctly parsed into five distinct panels, and that the download feature correctly exports the generated story as a text file.

8.1.2 Output Structure Testing

Given that the AI-generated response is free-form text, testing was conducted to confirm that the parsing logic in `comic_app.py` reliably identifies and separates the scene, action, dialogue and image-prompt sections for each of the five panels across a variety of different story ideas and themes, and handles minor variations in the model's response formatting gracefully.

8.1.3 Character Consistency Testing

Multiple test story ideas were submitted to qualitatively evaluate whether character names, descriptions, and personalities remained consistent from the first panel to the fifth panel, confirming the effectiveness of the character consistency prompt design described in Chapter 4.

8.1.4 Error Handling Testing

Testing was performed to confirm that the application handles error conditions gracefully, such as a missing or invalid `GEMINI_API_KEY`, network connectivity issues when contacting the Gemini API, and empty or extremely short user input, displaying an informative message to the user rather than an unhandled application crash.

8.2 Test Cases

Test Case ID	Description	Expected Result	Status
TC-01	Submit a valid story idea	Five-panel comic story generated and displayed	Pass
TC-02	Submit an empty story idea	Validation message shown; no API call made	Pass
TC-03	Run application with missing	Clear configuration error displayed	Pass

TC-04	GEMINI_API_KEY	to user	Pass
TC-05	Generate story and click Download	Text file with complete story downloaded	Pass
TC-06	Submit multiple different story ideas sequentially	Each generates an independent, correctly parsed story	Pass
TC-07	Review generated panels for character consistency	Character names/traits consistent across all panels	Pass
TC-08	Submit a very long or complex story idea	Application handles gracefully without crashing	Pass

Table 8.1: Test Cases and Results

Chapter 9: Results, Discussion, Conclusion and Future Enhancements

9.1 Results and Discussion

ComicCraft was successfully implemented and demonstrated to reliably transform short user-provided story ideas into structured, five-panel comic narratives, complete with scene descriptions, character actions, dialogues, and image prompts. The single-file Streamlit architecture proved highly effective for this use case, enabling rapid development and an intuitive, distraction-free user experience.

The character consistency mechanism, implemented purely through careful prompt engineering rather than any explicit memory or state-tracking system, was found to be effective across a range of test story ideas, producing coherent narratives where characters retained their identity and personality from the first to the fifth panel.

The separation of each panel's content into distinct scene, action, dialogue, and image-prompt fields proved valuable both for readability within the Streamlit interface and for the practical usability of the output — the generated image prompts, in particular, provide a ready-made bridge toward future integration with image-generation tools or use by human illustrators.

Overall, the results confirm that a lightweight, single-service architecture combining Streamlit and the Google Gemini API is well suited to building accessible, creative AI tools, achieving the objectives outlined in Chapter 1 with a minimal technology footprint.

9.2 Advantages

- Extremely simple deployment and setup, requiring only a Python environment, a small set of dependencies, and a Gemini API key.
- Structured, panel-by-panel output that is directly usable for comic adaptation, rather than unstructured prose.
- Effective character consistency across panels, achieved without complex state-management infrastructure.
- Ready-made image prompts for each panel, bridging narrative generation and future visual illustration.
- Accessible, no-technical-knowledge-required interface suitable for students, hobbyists and creative writers.
- Low operating cost, relying on the free tier of the Gemini API and free/low-cost Streamlit hosting options.

9.3 Limitations

- The application does not currently generate actual comic artwork; it produces

text-based image prompts that require a separate tool or artist to realize visually.

- As with any LLM-based system, occasional inconsistencies or lower-quality output can occur, particularly for very unusual or ambiguous story ideas.
- The application does not persist generated stories server-side; users must download their results, as there is no account system or story history.
- The fixed five-panel format, while effective for short stories, does not accommodate longer, multi-page comic narratives in the current version.

9.4 Conclusion

This project presented the design, architecture, and implementation of ComicCraft, an AI-powered comic story creator that uses Google's Gemini model to transform simple story ideas into structured, five-panel comic narratives. By combining the Streamlit framework with the Google GenAI SDK, the project demonstrates how a lightweight, single-service application can deliver a genuinely useful and accessible creative AI tool with minimal engineering overhead.

The application successfully fulfills its stated objectives: generating coherent, structured comic stories with distinct scenes, character actions, dialogues, and image prompts, while maintaining character consistency across panels, all through a simple and user-friendly web interface. The project illustrates the value of careful prompt engineering as a substitute for more complex state-management or fine-tuning approaches when working with capable, general-purpose Large Language Models.

ComicCraft serves as a practical demonstration of applying modern Generative AI to the creative writing and visual storytelling domain, and provides a solid, extensible foundation for a more comprehensive AI-assisted comic creation platform.

9.5 Future Enhancements

- Integrated image generation: extending the application to automatically generate corresponding artwork for each panel using an image-generation model, using the existing image prompts as direct input.
- Variable panel counts: allowing users to choose the number of panels or pages in their generated comic, rather than a fixed five-panel structure.
- Story history and accounts: introducing user accounts and persistent storage (for example, using a lightweight database) to allow users to save, revisit, and edit previously generated stories.
- Export formats: adding support for exporting generated stories as PDF or formatted comic-script documents, in addition to the current plain text download.
- Style and genre customization: allowing users to specify a desired art style, genre, or tone (for example, superhero, slice-of-life, or fantasy) to further guide the generation process.

- Collaborative editing: enabling multiple users to collaboratively refine a generated story before finalizing it.
- Deployment as a hosted service: packaging and deploying ComicCraft on a managed cloud platform or Streamlit Community Cloud for public access beyond local development use.

References

- [1] Streamlit Official Documentation — <https://docs.streamlit.io>
- [2] Google Gemini API Documentation — <https://ai.google.dev>
- [3] Google GenAI SDK Documentation — <https://ai.google.dev/gemini-api/docs/sdks>
- [4] python-dotenv Documentation — <https://pypi.org/project/python-dotenv/>
- [5] Python Official Documentation — <https://docs.python.org/3/>
- [6] ComicCraft Project Repository — GitHub: github.com/balsslaba7-cpu/ComicCraft

Appendix A: Sample requirements.txt

The following is a representative dependency manifest consistent with the technology stack described in Chapter 5, used to reproduce the ComicCraft environment via a single pip command.

```
streamlit>=1.30.0
google-genai>=0.3.0
python-dotenv>=1.0.0
```

Appendix B: Glossary

LLM (Large Language Model): A machine learning model trained on large volumes of text to generate and understand natural language; Gemini is the LLM used in ComicCraft.

Prompt Engineering: The practice of carefully designing the input text sent to a language model to reliably obtain a desired structure or quality of output.

Streamlit: An open-source Python framework for building interactive web applications with minimal front-end code.

Google GenAI SDK: The official Python software development kit used to authenticate and communicate with Google's Gemini models.

Character Consistency: The property of a generated narrative in which a character's name, appearance, and personality remain stable across multiple panels or scenes.

Image Prompt: A short descriptive text produced for a panel, intended to be used as input to an external image-generation model or as a brief for an illustrator.

Appendix C: Illustrative Prompt Template

The prompt template below illustrates the structure described in Sections 4.2.3 and 4.2.5, showing how a single instruction elicits a five-panel, character-consistent story from the Gemini model.

```
You are a comic story writer. Using the idea below, write a 5-panel
comic story. Keep character names, appearance and personality
identical across all panels. For EACH panel, output exactly:
Scene: <setting, time, and visual context>
Action: <what the characters do>
Dialogue: <spoken lines or captions>
Image Prompt: <a short visual description for an illustrator>
Story idea: "{user_input}"
```

This single structured instruction is the primary mechanism through which ComicCraft achieves both panel structure and character consistency, without any external state-tracking logic.

Appendix D: Frequently Asked Questions

Q: Does ComicCraft generate actual comic artwork?

A: No. The current version generates text-based image prompts for each panel; these prompts are designed to be used with a separate image-generation tool or by a human illustrator, as noted in Section 1.4 and Section 9.3.

Q: What happens if the Gemini API key is missing?

A: The application displays a clear configuration error to the user instead of crashing, as verified in test case TC-03 (Table 8.1).

Q: Can the number of panels be changed from five?

A: Not in the current version; a fixed five-panel format is used, though variable panel counts are listed as a future enhancement in Section 9.5.

Q: Is any user data stored on a server?

A: No. ComicCraft does not use a persistent database; each session is stateless and results are only available via direct file download, as described in Section 5.4.

Q: Which operating systems are supported?

A: Windows 10/11, macOS, and Linux, provided Python 3.9 or later is installed, as listed in Table 3.1.